Contents

Below is a proposed appendix chapter that treats the mortgage calculator as a **hybrid AI system**: deterministic financial computation is handled by conventional software, while the LLM provides the natural-language interface and explanation layer. That makes it a particularly good small example of the engineering principles developed throughout the bootcamp.

Appendix — Hybrid Mortgage Calculator MVP

A.1 Purpose

The hybrid mortgage calculator is a deliberately small AI engineering project that demonstrates an important architectural principle:

> **Use deterministic software for deterministic problems, and use AI where language understanding and explanation are valuable.**

The application combines a conventional mortgage calculator with a natural-language interface.

At its core, the calculator behaves like a regular financial calculator. Given any three of the following four quantities:

1. **Principal** — (P)
2. **Periodic interest rate** — (r)
3. **Total number of payments** — (n)
4. **Fixed periodic payment** — (M)

the system calculates the fourth.

For a conventional fixed-rate mortgage with monthly payments:

$$M = P\frac{r(1+r)^n}{(1+r)^n - 1}$$

where:

- (P) is the initial principal
- (r) is the monthly interest rate expressed as a decimal
- (n) is the total number of monthly payments
- (M) is the fixed monthly payment

The application also accepts natural-language questions such as:

> "What would the monthly payment be on a $500,000 mortgage at 6.5% for 30 years?"

or:

> "How much would I need to borrow if I can afford $3,000 a month at 6% for 30 years?"

The LLM interprets the question, extracts the parameters, invokes the deterministic calculator, and presents the result in natural language.

The LLM **does not perform the financial calculation itself**.

A.2 Why This Is a Hybrid AI Application

A naive implementation would send the user's question directly to an LLM:

```
What is the payment on a $500,000 mortgage at 6.5% for 30 years?
```

and accept the generated answer.

That is precisely the architecture we want to avoid.

Mortgage calculations are deterministic. There is no reason to use a probabilistic model to perform arithmetic that can be implemented exactly in a few lines of code.

Instead, the application separates the problem into two domains.

Deterministic domain

Conventional software performs:

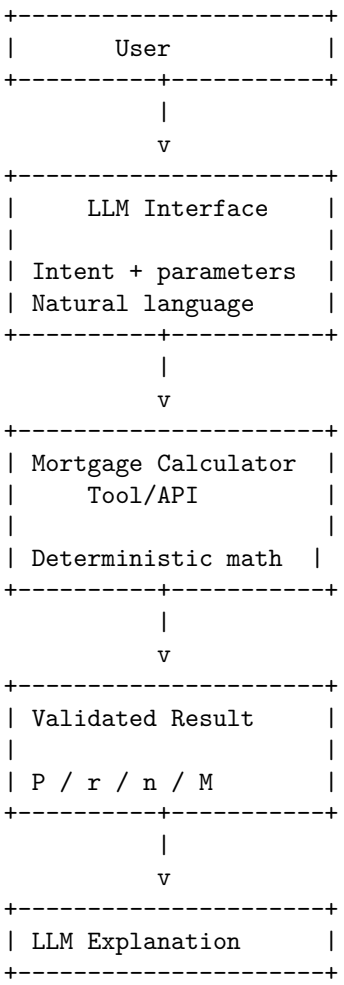
- financial calculations
- parameter validation
- unit conversion
- rounding
- numerical error handling
- amortization calculations
- consistency checks

Probabilistic domain

The LLM performs:

- natural-language understanding
- parameter extraction
- intent classification
- conversational interaction
- explanation
- clarification of ambiguous questions

The resulting architecture is:



This architecture provides an important engineering property:

> **The AI can be wrong about language, but it cannot change the underlying financial calculation.**

A.3 MVP Scope

The MVP intentionally has a narrow scope.

Required capabilities

The application must support:

- direct calculator mode
- natural-language question mode
- fixed-rate mortgages
- monthly payments
- calculation of exactly one missing quantity
- validation of supplied parameters
- deterministic numerical results
- natural-language explanations
- basic amortization information

The four primary variables are:

Variable	Symbol	Example
Principal	(P)	$500,000
Monthly interest rate	(r)	0.005
Number of payments	(n)	360
Monthly payment	(M)	$3,160.34

The user supplies any three.

The application calculates the fourth.

A.4 The Four Core Calculations

The calculator must implement all four inverse relationships.

A.4.1 Calculate Payment

Given:

$$P,\ r,\ n$$

calculate:

$$M = P\frac{r(1+r)^n}{(1+r)^n - 1}$$

For example:

```
Principal:       $500,000
Annual rate:       6.5%
Term:              30 years
```

The monthly rate is:

$$r = \frac{0.065}{12}$$

and:

$$n = 30 \times 12 = 360$$

The calculator returns the fixed monthly payment.

A.4.2 Calculate Principal

Given:

$$M,\ r,\ n$$

calculate:

$$P = M\frac{(1+r)^n - 1}{r(1+r)^n}$$

Equivalently:

$$P = M\frac{1-(1+r)^{-n}}{r}$$

Example:

```
Monthly payment:  $3,000
Annual rate:          6%
Term:                30 years
```

The calculator determines the maximum principal corresponding to that payment.

A.4.3 Calculate Number of Payments

Given:

$$P,\ r,\ M$$

calculate:

$$n = \frac{\ln\left(\frac{M}{M-Pr}\right)}{\ln(1+r)}$$

This calculation requires:

$$M > Pr$$

because (Pr) is the interest charged during the first period.

If:

$$M \leq Pr$$

the loan cannot be fully amortized with the specified payment.

The calculator must detect this condition rather than returning a meaningless result.

A.4.4 Calculate Interest Rate

Given:

$$P,\ M,\ n$$

calculate (r).

Unlike the other three calculations, there is generally no simple closed-form solution for (r).

The application therefore solves:

$$M = P\frac{r(1+r)^n}{(1+r)^n - 1}$$

numerically.

The MVP can use a robust numerical method such as:

- bisection
- Newton-Raphson
- Brent's method

Bisection is particularly attractive for an MVP because it is simple, deterministic, and robust.

The numerical solver should have:

- a defined search interval
- convergence tolerance
- maximum iteration count
- failure handling

For example:

```
lower rate = 0
upper rate = 1
tolerance  = 1e-12
max_iters  = 100
```

The exact implementation should be isolated behind the calculator API so that it can later be replaced without changing the application interface.

A.5 Input Model

The calculator should use an explicit structured input model.

For example:

```
{
  "principal": 500000,
  "periodic_rate": 0.0054166667,
  "payments": 360,
  "payment": null,
  "payment_frequency": "monthly"
}
```

The application determines which field is missing.

The internal representation should use normalized units.

For example:

```
$500,000             → 500000
6.5% annual          → 0.065 annual
6.5% monthly         → 0.0054166667 monthly
30 years             → 360 payments
```

This separation between **human representation** and **computational representation** is important.

The user may say:

> "6.5 percent for thirty years."

The calculator should receive:

```
annual_rate = 0.065
payments = 360
```

rather than attempting to reason about natural-language expressions throughout the numerical code.

A.6 Calculation Engine

The calculation engine should be implemented as a conventional, independently testable module.

Conceptually:

```
calculate(
    principal=None,
    periodic_rate=None,
    payments=None,
    payment=None
)
```

The function should enforce the invariant:

> Exactly one of the four quantities must be missing.

Examples:

```
P + r + n → calculate M
M + r + n → calculate P
P + M + n → calculate r
P + r + M → calculate n
```

Invalid cases include:

```
Only two parameters supplied
All four parameters supplied
No parameters supplied
```

The calculator should reject these explicitly.

It should never guess which value the user intended to omit.

A.7 Financial Validation

Validation belongs in deterministic code, not in the LLM.

Examples include:

Principal

$$P > 0$$

Interest rate

For the basic MVP:

$$r \geq 0$$

Number of payments

$$n > 0$$

and normally:

$$n \in \mathbb{Z}$$

Payment

$$M > 0$$

Additional financial constraints should be checked according to the calculation being performed.

For example, when solving for the term:

$$M > Pr$$

must hold.

The system should return structured errors rather than natural-language error messages from the calculation layer.

Example:

```
{
  "error": "PAYMENT_TOO_LOW",
  "message": "Payment does not exceed periodic interest.",
  "parameter": "payment"
}
```

The LLM can subsequently turn that structured error into an understandable explanation.

A.8 Natural-Language Interface

The natural-language interface is where the LLM adds value.

Consider:

> "I want to buy a $600,000 house. I'll put 20% down. If the mortgage rate is 6.25%, what will my payment be over 30 years?"

The LLM must infer:

```
purchase price = $600,000
down payment = 20%
principal = $480,000
annual rate = 6.25%
term = 30 years
payments = 360
missing quantity = monthly payment
```

It then invokes the deterministic calculation tool.

The LLM should not calculate the payment itself.

A.9 Tool Interface

The LLM should interact with the calculator through a structured tool.

Conceptually:

```
{
  "name": "calculate_mortgage",
  "description": "Calculate exactly one missing mortgage parameter from the other three.",
  "parameters": {
    "principal": "number | null",
    "periodic_rate": "number | null",
    "payments": "integer | null",
    "payment": "number | null",
    "payment_frequency": "monthly"
  }
}
```

The tool returns structured data.

For example:

```
{
  "principal": 500000,
  "periodic_rate": 0.0054166667,
  "payments": 360,
```

```
  "payment": 3159.38,
  "annual_rate": 0.065,
  "term_years": 30,
  "total_paid": 1137376.80,
  "total_interest": 637376.80
}
```

The LLM then explains the result.

This is an instance of a general AI engineering pattern:

```
Natural language
       ↓
Structured intent
       ↓
Deterministic tool
       ↓
Structured result
       ↓
Natural-language response
```

A.10 System Prompt

The LLM's system instructions should establish a strict boundary between interpretation and computation.

A conceptual prompt might state:

```
You are the natural-language interface for a fixed-rate mortgage calculator.

Your responsibilities are:

1. Understand the user's mortgage question.
2. Extract the relevant numerical parameters.
3. Convert units into the calculator's canonical representation.
4. Ask for clarification when required information is missing or ambiguous.
5. Call the mortgage calculation tool for all numerical calculations.
6. Never perform mortgage arithmetic yourself when the calculator tool can do it.
7. Explain calculator results clearly.
8. State assumptions explicitly.
9. Do not invent financial parameters.
10. Distinguish principal-and-interest payment from taxes,
    insurance, HOA fees, and other housing costs.

The calculator supports fixed-rate mortgages with periodic payments.
```

The critical instruction is not the wording itself but the architectural contract:

> **LLM = interpreter and explainer. Calculator = authority.**

A.11 Conversational Examples

Example 1 — Straightforward question

User:

> "What's the payment on a $500,000 mortgage at 6.5% for 30 years?"

LLM extracts:

```
P = 500000
annual rate = 0.065
n = 360
```

The tool calculates (M).

The response might be:

> On a $500,000 fixed-rate mortgage at 6.5% for 30 years, the principal-and-interest payment is approximately $3,160 per month.

Example 2 — Solve for principal

User:

> "I can afford $3,000 a month. How much could I borrow at 6% for 30 years?"

The LLM extracts:

```
M = 3000
annual rate = 0.06
n = 360
```

The missing quantity is (P).

The tool calculates it.

Example 3 — Ambiguous question

User:

> "How much would a $500,000 mortgage cost me?"

This is underspecified.

The LLM should not guess the interest rate or term.

It should ask:

> What interest rate and mortgage term would you like me to use? For example, I can calculate it at 6.5% for 30 years.

This demonstrates an important agent behavior:

> **Ask for missing information rather than hallucinating it.**

Example 4 — Financially invalid request

User:

> "I owe $500,000 at 6% and can pay $2,000 a month. How long will it take?"

The calculator determines that:

$$Pr = 500000 \times 0.005 = 2500$$

The payment is only $2,000.

Therefore:

$$M < Pr$$

and the loan cannot amortize.

The LLM should explain:

> At 6%, the first month's interest alone is $2,500, which is greater than your $2,000 payment. Under these assumptions, the balance would increase rather than being paid off.

The explanation is generated by the LLM, but the underlying conclusion comes from the deterministic calculator.

A.12 Amortization

A useful MVP extension is an amortization schedule.

For each payment period:

$$I_t = B_{t-1} r$$

where (I_t) is the interest portion and (B_{t-1}) is the previous balance.

The principal portion is:

$$A_t = M - I_t$$

and the new balance is:

$$B_t = B_{t-1} - A_t$$

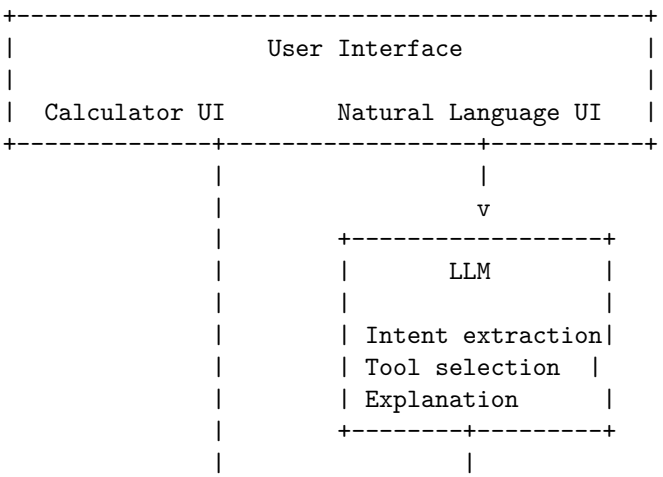
The application can therefore produce:

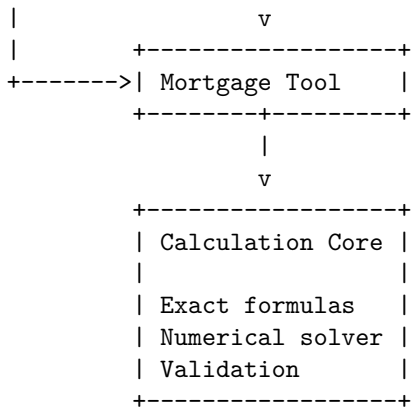
Payment	Payment	Principal	Interest	Balance
1	$3,160	$452	$2,708	$499,548
2	…	…	…	…
360	…	…	…	$0

The LLM can answer questions about this schedule, but the schedule itself should be generated deterministically.

A.13 Architecture

The MVP can be implemented as five logical components.





A simple implementation could use:

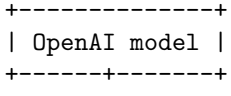
```
Frontend:
    Streamlit / React / simple HTML

Application:
    Python

LLM:
    API model or local model

Tool interface:
    Function calling / structured output

Calculation:
    Pure Python

Tests:
    pytest
```

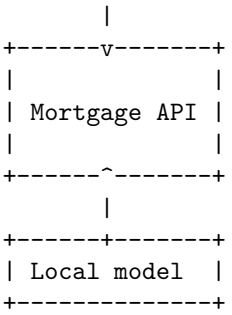
The architecture should remain independent of any particular model provider.

A.14 Model Independence

The MVP should make it possible to replace the LLM without changing the financial engine.

For example:





The calculation core remains identical.

This provides a useful demonstration of **model substitution** as an architectural principle.

A.15 Testing Strategy

The project should demonstrate the distinction between traditional software tests and AI evaluation.

Deterministic tests

The calculation engine should have comprehensive unit tests.

Test:

- payment calculation
- principal calculation
- rate calculation
- term calculation
- zero-interest loans
- boundary values
- invalid inputs
- numerical convergence
- rounding
- amortization schedules

Known-value test cases should be included.

For example:

```
P = 100,000
annual rate = 6%
term = 30 years
```

The expected payment can be computed independently and used as a regression test.

A.16 Property-Based Testing

Mortgage mathematics is particularly well suited to property-based testing.

For randomly generated valid loans:

```
(P, r, n)
        ↓
calculate M
        ↓
calculate P'
```

The resulting:

$$P' \approx P$$

should hold within a defined numerical tolerance.

Similarly:

```
(P, r, n)
        ↓
M
        ↓
solve for r
```

should return approximately the original (r).

These tests validate the relationships between the four inverse calculations rather than merely testing a collection of fixed examples.

A.17 LLM Evaluation

The natural-language interface requires a different evaluation strategy.

Create a dataset containing questions such as:

```
"What is the payment on a $400k loan at 6.25% for 30 years?"

"I can pay $2,500/month. How much can I borrow at 6%?"

"How many years would it take to pay off $300,000 at 5.5% if I pay $2,000?"

"What rate am I effectively paying if I borrow $400k and make
$2,500 payments for 30 years?"
```

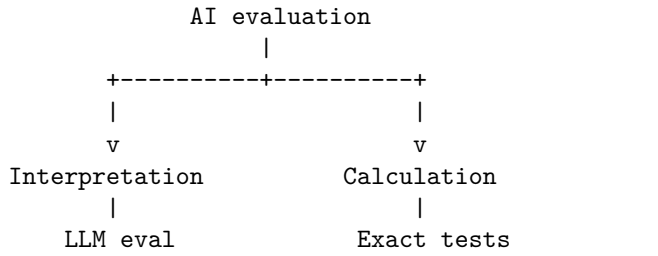
```
"What would the payment be?"
```

Evaluate whether the LLM correctly identifies:

- intent
- principal
- rate
- rate period
- number of payments
- payment
- missing quantity
- assumptions
- need for clarification

The actual numerical answer should be evaluated against the deterministic calculator rather than against another LLM.

This produces a clean evaluation decomposition:



A.18 Error Taxonomy

The project should explicitly classify failures.

Type 1 — Interpretation error

The user says:

> "6.5% for 30 years"

and the LLM incorrectly interprets 6.5% as a monthly rate.

Type 2 — Unit conversion error

The model interprets:

> "30 years"

as:

30

rather than:

360

monthly payments.

Type 3 — Missing information

The user asks for a payment without providing a rate.

The system should ask for clarification.

Type 4 — Calculation error

The calculator itself produces the wrong result.

This is a conventional software defect and should be caught by tests.

Type 5 — Explanation error

The calculator produces the correct result but the LLM explains it incorrectly.

Type 6 — Scope error

The user asks:

> "What will my total housing payment be?"

but the application only calculates principal and interest.

The LLM must not silently invent property taxes, insurance, HOA fees, or other costs.

A.19 Guardrails

The application should explicitly constrain the LLM.

The model should:

1. Never invent an interest rate.
2. Never invent a loan term.
3. Never substitute annual and periodic rates without conversion.
4. Never calculate the result independently when the calculator is available.
5. Never override calculator validation.
6. Clearly distinguish principal and interest from total housing expenses.

7. State assumptions.
8. Identify when a question is outside the application's scope.
9. Avoid presenting estimates as lender-specific quotes.
10. Preserve the calculator's numerical precision until final presentation.

These are examples of **application-level AI safety**, rather than relying exclusively on model behavior.

A.20 Precision and Rounding

Financial calculations require particular care with rounding.

The internal calculation should use full floating-point precision or an appropriate decimal representation.

Rounding should occur at the presentation boundary.

For example:

```
Internal:
3159.384721...

Displayed:
$3,159.38
```

The application should avoid repeatedly rounding intermediate values because this can accumulate errors in amortization calculations.

The MVP should document its rounding policy explicitly.

A.21 Scope Boundaries

The MVP is intentionally a **fixed-rate mortgage calculator**, not a complete mortgage underwriting system.

It should not initially attempt to model:

- adjustable-rate mortgages
- interest-only loans
- balloon payments
- negative amortization
- refinancing costs
- closing costs
- mortgage insurance
- property taxes
- homeowners insurance

- HOA fees
- lender-specific fees
- points
- escrow
- tax deductions
- jurisdiction-specific regulations

These could become future extensions.

Keeping them outside the MVP is important because it prevents the natural-language interface from creating the illusion that the application supports financial concepts that the calculation engine does not actually model.

A.22 Suggested Project Structure

A minimal Python implementation might look like:

```
mortgage_calculator/
|
+-- pyproject.toml
+-- README.md
|
+-- src/
|   +-- mortgage/
|       +-- __init__.py
|       +-- calculator.py
|       +-- validation.py
|       +-- amortization.py
|       +-- models.py
|       +-- llm.py
|       +-- prompts.py
|
+-- tests/
|   +-- test_payment.py
|   +-- test_principal.py
|   +-- test_rate.py
|   +-- test_term.py
|   +-- test_amortization.py
|   +-- test_validation.py
|   +-- test_nl_interface.py
|
+-- evals/
    +-- mortgage_questions.jsonl
    +-- evaluate.py
```

The important architectural boundary is:

```
llm.py
   |
   v
calculator.py
   |
   v
mathematical functions
```

The calculation module should have no dependency on the LLM.

A.23 MVP Development Sequence

The project should be built incrementally.

Step 1 — Mathematical core

Implement:

- payment
- principal
- term
- rate

with no AI.

Step 2 — Validation

Add:

- input validation
- domain constraints
- structured errors

Step 3 — Tests

Add:

- unit tests
- known-value tests
- inverse/property tests

At this point the mortgage calculator should be trustworthy independently of AI.

Step 4 — Calculator UI

Build a conventional form:

```
Principal:       [ $500,000 ]
Interest rate:   [ 6.50%    ]
Term:            [ 30 years ]
Payment:         [ CALCULATE ]
```

Step 5 — Tool interface

Expose the deterministic calculator as a structured function.

Step 6 — LLM interface

Add natural-language interpretation and tool calling.

Step 7 — Evaluation

Create a natural-language evaluation dataset and measure:

- parameter extraction accuracy
- tool-selection accuracy
- clarification accuracy
- numerical correctness
- explanation quality

Step 8 — Hardening

Test:

- ambiguous questions
- malformed values
- unusual units
- adversarial prompts
- unsupported mortgage types
- impossible loans
- model failures
- calculator failures

A.24 What Makes This an AI Engineering Project?

At first glance, this project appears to be nothing more than a mortgage calculator with a chatbot attached.

That interpretation misses the important lesson.

The project demonstrates several fundamental AI systems engineering principles.

1. Probabilistic and deterministic components have different jobs

The LLM handles language.

The calculator handles mathematics.

2. Tools create reliability boundaries

The LLM does not need to be trusted with arithmetic when it can invoke a deterministic function.

3. Structured interfaces reduce ambiguity

Natural language is converted into a typed representation before computation.

4. Validation belongs below the model

Business and mathematical invariants are enforced by software.

5. Evaluation must follow the architecture

The calculation engine is tested conventionally.

The language interface is evaluated probabilistically.

6. Model quality is not the same as system quality

A better model may improve parameter extraction, but it cannot compensate for a defective calculation engine.

7. The system should fail explicitly

When information is missing, the correct behavior is clarification—not hallucination.

A.25 Extensions

Once the MVP is reliable, the project can evolve into a richer AI application.

Potential extensions include:

Comparative scenarios

> "Compare a 15-year and 30-year mortgage."

Extra payments

> "What happens if I pay an extra $500 per month?"

Refinancing analysis

> "Would refinancing from 7% to 5.5% save me money?"

Amortization questions

> "How much interest will I have paid after ten years?"

Sensitivity analysis

> "How much does my payment change for every 0.25% change in the interest rate?"

Natural-language scenario construction

> "I have $150,000 for a down payment and want to keep my mortgage payment below $3,500. What homes could I afford at 6%?"

The final example illustrates an important architectural transition.

The system is no longer merely a calculator. It is becoming a **tool-using financial reasoning application**, in which deterministic financial primitives are composed into higher-level workflows.

A.26 The Central Design Lesson

The hybrid mortgage calculator is intentionally small, but its architecture scales.

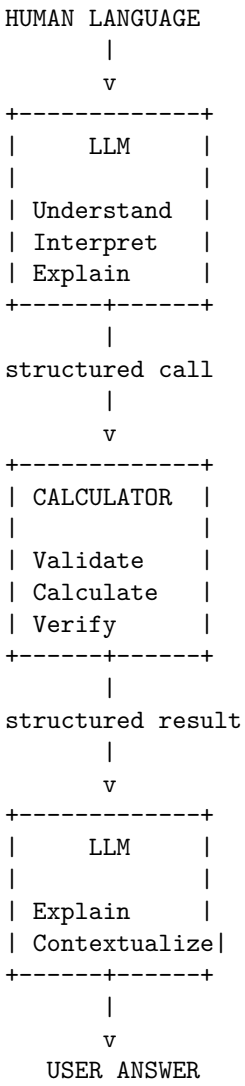
A common misconception about AI applications is:

> "The AI should do the work."

A better engineering principle is:

> **The AI should orchestrate the work that requires intelligence, while conventional software performs the work for which conventional software is better suited.**

For the mortgage calculator:



This is the essence of the hybrid approach.

The LLM provides the flexible, probabilistic interface that makes the application conversational. The deterministic calculation engine provides the reliability required for numerical correctness.

The result is not an **LLM-powered mortgage calculator**.

It is a **mortgage calculator with an LLM interface**.

That distinction is small in wording but fundamental in AI systems engineering.